

30. Custom Events Dispatch Karna

Events sirf sunne ke liye nahi, **khud generate** bhi kar sakte ho. Custom events se apne components (menu, slider) batate hain ki andar kya ho raha hai (`open`, `select`, ...). Built-in events (`click`) generate karna automated testing me kaam aata hai.

Event constructor

js

```
let event = new Event(type, { bubbles: true/false, cancelable: true/false });
```

- `bubbles`: event bubble ho ya nahi (default `false`)
- `cancelable`: `preventDefault()` kaam kare ya nahi (default `false`)

dispatchEvent

Event object banane ke baad element par "chalao":

js

```
let event = new Event("click");  
elem.dispatchEvent(event);
```

Handlers aise react karte hain jaise asli event hua ho.

`event.isTrusted`: asli user event ke liye `true`, script se bane event ke liye `false`.

Bubbling example

js

```
document.addEventListener("hello", function(event) {  
  alert("Hello from " + event.target.tagName);  
});  
  
elem.dispatchEvent(new Event("hello", { bubbles: true }));
```

- Custom events ke liye `addEventListener` hi use karo (`document.onhello` nahi chalta).
- `bubbles: true` na do to event upar nahi jaayega.

`MouseEvent`, `KeyboardEvent`, etc.

Built-in UI events ke liye sahi class use karo, taaki specific properties de sako:

js

```
let event = new MouseEvent("click", {
  bubbles: true, cancelable: true, clientX: 100, clientY: 100
});
event.clientX; // 100
```

Generic `new Event` me `clientX` jaisi properties ignore ho jaati hain.

CustomEvent (apne naye events ke liye)

Extra data `detail` me bhejo:

js

```
elem.addEventListener("hello", function(event) {
  alert(event.detail.name);
});
elem.dispatchEvent(new CustomEvent("hello", {
  detail: { name: "John" }
}));
```

Custom events me `preventDefault()`

Custom events ka koi browser default action nahi hota, par dispatch karne wala code apna action cancel karwa sakta hai:

js

```
function hide() {
  let event = new CustomEvent("hide", { cancelable: true }); // cancelable zar
  if (!rabbit.dispatchEvent(event)) {
    alert('Prevented by a handler'); // dispatchEvent false lautata hai
  } else {
    rabbit.hidden = true;
  }
}
rabbit.addEventListener('hide', function(event) {
  if (confirm("Call preventDefault?")) event.preventDefault();
});
```

Events ke andar events synchronous hote hain

Normal events queue me lagte hain, par agar ek event ke handler ke andar se

`dispatchEvent` karo to **turant** chalta hai:

js

```
menu.onclick = function() {  
  alert(1);  
  menu.dispatchEvent(new CustomEvent("menu-open", { bubbles: true }));  
  alert(2);  
};  
document.addEventListener('menu-open', () => alert('nested'));  
// Output: 1 -> nested -> 2
```

Baad me chalana ho to `dispatchEvent` ko `setTimeout(() => ..., 0)` me wrap karo (output: 1 -> 2 -> nested) ya handler ke aakhir me rakho.

Summary

- Naya event: `new Event(name, {bubbles, cancelable})`, custom data ke liye `new CustomEvent(name, {detail})`.
- Specific type ke liye `MouseEvent`, `KeyboardEvent` jaisi classes.
- Chalao: `elem.dispatchEvent(event)`.
- Browser events (`click`) hack ke roop me generate karna aam taur par bad architecture hai. Sirf testing ya 3rd-party library ko chalane ke liye. Apne custom events architecture ke liye acche hain.